JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class LockSupport

java.lang.Object
    java.util.concurrent.locks.LockSupport

```
public final class LockSupport
extends Object
```

Basic thread blocking primitives for creating locks and other synchronization classes.

This class associates, with each thread that uses it, a permit (in the sense of the `Semaphore` class). A call to `park` will return immediately if the permit is available, consuming it in the process; otherwise it *may* block. A call to `unpark` makes the permit available, if it was not already available. (Unlike with Semaphores though, permits do not accumulate. There is at most one.) Reliable usage requires the use of volatile (or atomic) variables to control when to park or unpark. Orderings of calls to these methods are maintained with respect to volatile variable accesses, but not necessarily non-volatile variable accesses.

Methods `park` and `unpark` provide efficient means of blocking and unblocking threads
that do not encounter the problems that cause the deprecated methods `Thread.suspend` and `Thread.resume` to be unusable for such purposes: Races between one thread invoking `park` and another thread trying to `unpark` it will preserve liveness, due to the permit. Additionally, `park` will return if the caller's thread was interrupted, and timeout versions are supported. The `park` method may also return at any other time, for "no reason", so in general must be invoked within a loop that rechecks conditions upon return. In this sense `park` serves as an optimization of a "busy wait" that does not waste as much time spinning, but must be paired with an `unpark` to be effective.

The three forms of `park` each also support a `blocker` object parameter. This object is recorded while the thread is blocked to permit monitoring and diagnostic tools to identify the reasons that threads are blocked. (Such tools may access blockers using method `getBlocker(Thread)`.) The use of these forms rather than the original forms without this parameter is strongly encouraged. The normal argument to supply as a `blocker` within a lock implementation is `this`.

These methods are designed to be used as tools for creating higher-level synchronization utilities, and are not in themselves useful for most concurrency control applications. The `park` method is designed for use only in constructions of the form:

```
while (!canProceed()) {
  // ensure request to unpark is visible to other threads
  ...
  LockSupport.park(this);
}
```

where no actions by the thread publishing a request to unpark, prior to the call to `park`, entail locking or blocking. Because only one permit is associated with each thread, any intermediary uses of `park`, including implicitly via class loading, could lead to an unresponsive thread (a "lost unpark").

Sample Usage. Here is a sketch of a first-in-first-out non-reentrant lock class:

```
class FIFOMutex {
  private final AtomicBoolean locked = new AtomicBoolean(false);
  private final Queue<Thread> waiters
    = new ConcurrentLinkedQueue<>();

  public void lock() {
    boolean wasInterrupted = false;
    // publish current thread for unparkers
    waiters.add(Thread.currentThread());

    // Block while not first in queue or cannot acquire lock
    while (waiters.peek() != Thread.currentThread() ||
           !locked.compareAndSet(false, true)) {
      LockSupport.park(this);
      // ignore interrupts while waiting
      if (Thread.interrupted())
        wasInterrupted = true;
    }

    waiters.remove();
    // ensure correct interrupted status on return
    if (wasInterrupted)
      Thread.currentThread().interrupt();
  }

  public void unlock() {
    locked.set(false);
    LockSupport.unpark(waiters.peek());
  }
```

```
  static {
    // Reduce the risk of "lost unpark" due to classloading
    Class<?> ensureLoaded = LockSupport.class;
  }
}
```

Since:

1.5

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
static Object	**getBlocker**(Thread t)	Returns the blocker object supplied to the most recent invocation of a park method that has not yet unblocked, or null if not blocked.
static void	**park**()	Disables the current thread for thread scheduling purposes unless the permit is available.
static void	**park**(Object blocker)	Disables the current thread for thread scheduling purposes unless the permit is available.
static void	**parkNanos**(long nanos)	Disables the current thread for thread scheduling purposes, for up to the specified waiting time, unless the permit is available.
static void	**parkNanos** (Object blocker, long nanos)	Disables the current thread for thread scheduling purposes, for up to the specified

		waiting time, unless the permit is available.
static void	**parkUntil**(long deadline)	Disables the current thread for thread scheduling purposes, until the specified deadline, unless the permit is available.
static void	**parkUntil**(Object blocker, long deadline)	Disables the current thread for thread scheduling purposes, until the specified deadline, unless the permit is available.
static void	**setCurrentBlocker**(Object blocker)	Sets the object to be returned by invocations of getBlocker for the current thread.
static void	**unpark**(Thread thread)	Makes available the permit for the given thread, if it was not already available.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Details

setCurrentBlocker

```
public static void setCurrentBlocker(Object blocker)
```

Sets the object to be returned by invocations of getBlocker for the current thread. This method may be used before invoking the no-argument version of park() from non-public objects, allowing more helpful diagnostics, or

retaining compatibility with previous implementations of blocking methods. Previous values of the blocker are not automatically restored after blocking. To obtain the effects of `park(b}`, use `setCurrentBlocker(b); park(); setCurrentBlocker(null);`

Parameters:

`blocker` - the blocker object

Since:

14

unpark

```
public static void unpark(Thread thread)
```

Makes available the permit for the given thread, if it was not already available. If the thread was blocked on `park` then it will unblock. Otherwise, its next call to `park` is guaranteed not to block. This operation is not guaranteed to have any effect at all if the given thread has not been started.

Parameters:

`thread` - the thread to unpark, or `null`, in which case this operation has no effect

park

```
public static void park(Object blocker)
```

Disables the current thread for thread scheduling purposes unless the permit is available.

If the permit is available then it is consumed and the call returns immediately; otherwise the current thread becomes disabled for thread scheduling purposes and lies dormant until one of three things happens:

- Some other thread invokes `unpark` with the current thread as the target; or
- Some other thread interrupts the current thread; or
- The call spuriously (that is, for no reason) returns.

This method does *not* report which of these caused the method to return. Callers should re-check the conditions which caused the thread to park in the first place. Callers may also determine, for example, the interrupted status of the thread upon return.

Parameters:

`blocker` - the synchronization object responsible for this thread parking

Since:

1.6

parkNanos

```
public static void parkNanos(Object blocker,
                             long nanos)
```

Disables the current thread for thread scheduling purposes, for up to the specified waiting time, unless the permit is available.

If the specified waiting time is zero or negative, the method does nothing. Otherwise, if the permit is available then it is consumed and the call returns immediately; otherwise the current thread becomes disabled for thread scheduling purposes and lies dormant until one of four things happens:

- Some other thread invokes `unpark` with the current thread as the target; or
- Some other thread interrupts the current thread; or
- The specified waiting time elapses; or
- The call spuriously (that is, for no reason) returns.

This method does *not* report which of these caused the method to return. Callers should re-check the conditions which caused the thread to park in the first place. Callers may also determine, for example, the interrupted status of the thread, or the elapsed time upon return.

Parameters:

`blocker` - the synchronization object responsible for this thread parking

`nanos` - the maximum number of nanoseconds to wait

Since:

1.6

parkUntil

```
public static void parkUntil(Object blocker,
                             long deadline)
```

Disables the current thread for thread scheduling purposes, until the specified deadline, unless the permit is available.

If the permit is available then it is consumed and the call returns immediately; otherwise the current thread becomes disabled for thread scheduling purposes and lies dormant until one of four things happens:

- Some other thread invokes `unpark` with the current thread as the target; or
- Some other thread interrupts the current thread; or

- The specified deadline passes; or
- The call spuriously (that is, for no reason) returns.

This method does *not* report which of these caused the method to return. Callers should re-check the conditions which caused the thread to park in the first place. Callers may also determine, for example, the interrupted status of the thread, or the current time upon return.

Parameters:

`blocker` - the synchronization object responsible for this thread parking

`deadline` - the absolute time, in milliseconds from the Epoch, to wait until

Since:

1.6

getBlocker

```
public static Object getBlocker(Thread t)
```

Returns the blocker object supplied to the most recent invocation of a park method that has not yet unblocked, or null if not blocked. The value returned is just a momentary snapshot -- the thread may have since unblocked or blocked on a different blocker object.

Parameters:

`t` - the thread

Returns:

the blocker

Throws:

`NullPointerException` - if argument is null

Since:

1.6

park

```
public static void park()
```

Disables the current thread for thread scheduling purposes unless the permit is available.

If the permit is available then it is consumed and the call returns immediately; otherwise the current thread becomes disabled for thread scheduling purposes and lies dormant until one of three things happens:

- Some other thread invokes `unpark` with the current thread as the target; or

- Some other thread interrupts the current thread; or
- The call spuriously (that is, for no reason) returns.

This method does *not* report which of these caused the method to return. Callers should re-check the conditions which caused the thread to park in the first place. Callers may also determine, for example, the interrupted status of the thread upon return.

parkNanos

```
public static void parkNanos(long nanos)
```

Disables the current thread for thread scheduling purposes, for up to the specified waiting time, unless the permit is available.

If the specified waiting time is zero or negative, the method does nothing. Otherwise, if the permit is available then it is consumed and the call returns immediately; otherwise the current thread becomes disabled for thread scheduling purposes and lies dormant until one of four things happens:

- Some other thread invokes `unpark` with the current thread as the target; or
- Some other thread interrupts the current thread; or
- The specified waiting time elapses; or
- The call spuriously (that is, for no reason) returns.

This method does *not* report which of these caused the method to return. Callers should re-check the conditions which caused the thread to park in the first place. Callers may also determine, for example, the interrupted status of the thread, or the elapsed time upon return.

Parameters:

`nanos` - the maximum number of nanoseconds to wait

parkUntil

```
public static void parkUntil(long deadline)
```

Disables the current thread for thread scheduling purposes, until the specified deadline, unless the permit is available.

If the permit is available then it is consumed and the call returns immediately; otherwise the current thread becomes disabled for thread scheduling purposes and lies dormant until one of four things happens:

- Some other thread invokes `unpark` with the current thread as the target; or
- Some other thread interrupts the current thread; or

- The specified deadline passes; or
- The call spuriously (that is, for no reason) returns.

This method does *not* report which of these caused the method to return. Callers should re-check the conditions which caused the thread to park in the first place. Callers may also determine, for example, the interrupted status of the thread, or the current time upon return.

Parameters:

`deadline` - the absolute time, in milliseconds from the Epoch, to wait until

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.